

Reviewer Pack — Tropical Cycle Rank and Communication Complexity

Entry 4695f49f0b2b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 17:32:34 UTC

Tropical Cycle Rank and Communication Complexity

Entry ID: 4695f49f0b2b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 17:32:34 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry × Boolean Function Complexity: Circuit Complexity **Field B** (complexity object): Communication Complexity for XOR

Statement:

For any boolean function f with communication complexity $CC_R(f)$ and associated tropical polynomial t_f , the tropical cycle rank of t_f is upper bounded by $2^{CC_R(f)}$. Equivalently, there exists a constant $c > 0$ such that for all boolean functions f with $CC_R(f) \leq n$, the tropical cycle rank of the associated tropical polynomial t_f satisfies $TR(t_f) \leq 2^n * c$.

Rationale (proposer's reasoning):

Tropical geometry provides a framework to study discrete objects through their tropicalizations. The tropical cycle rank is an invariant in tropical geometry that may capture essential properties of boolean functions. By linking this geometric concept with communication complexity, we could potentially expose new structural information about the complexity of boolean functions. If such a connection holds, it would suggest that geometric properties of boolean functions can be used to understand their computational complexity.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 663740a9a80da1f1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the tropical cycle rank ($TR(t_f)$) of the associated tropical polynomial t_f for each boolean function f satisfies $TR(t_f) \leq 2^{CC_R(f)}$ for all seeds, where $CC_R(f)$ is the communication complexity. The conjecture is falsified if any seed produces a value where $TR(t_f) > 2^{CC_R(f)}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical cycle rank" AND "communication complexity" AND boolean function" - "XOR communication complexity" AND "tropical geometry" AND circuit complexity" - "tropical polynomial" AND "cycle rank" AND $CC_R(f) \leq n$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tropical_polynomial(f):
        return [f(x) for x in range(2**len(f))]

    def communication_complexity(f):
        n = len(f)
        if n == 1:
            return 0
        cc = float('inf')
        for i in range(n):
            f_i = lambda x: f(x ^ (1 << i))
            cc_i = communication_complexity(f_i)
            if cc_i < cc:
                cc = cc_i
        return cc + 1

    def tropical_cycle_rank(t_f):
        n = len(t_f)
        if n == 0:
            return 0
        rank = 1
        for i in range(n):
            t_f_i = [t_f[j] ^ t_f[i] for j in range(n)]
            rank_i = tropical_cycle_rank(t_f_i)
```

```

        if rank_i > rank:
            rank = rank_i
    return rank

def boolean_function(n):
    return random.choices([0, 1], k=n)

n_max = 40
instances_tested = 0
total_metric_value = 0.0
conjecture_holds = True
counterexample = ""

for n in range(5, n_max + 1):
    f = boolean_function(n)
    cc_r_f = communication_complexity(f)
    t_f = tropical_polynomial(f)
    tr_t_f = tropical_cycle_rank(t_f)

    instances_tested += 1
    total_metric_value += tr_t_f

    if tr_t_f > 2**cc_r_f:
        conjecture_holds = False
        counterexample = f"CC_R(f)={cc_r_f}, TR(t_f)={tr_t_f}"

mean_metric_value = total_metric_value / instances_tested
std_metric_value = math.sqrt(sum((x - mean_metric_value)**2 for x in range(instances_tested)))

return {
    "metric_name": "tropical_cycle_rank",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])

```

```

    print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41de562a.py", line 88, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41de562a.py", line 59, in run_trial
    cc_r_f = communication_complexity(f)
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41de562a.py", line 31, in communication_complexity
    cc_i = communication_complexity(f_i)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41de562a.py", line 25, in communication_complexity
    n = len(f)
        ^^^^^
TypeError: object of type 'function' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture’s validity. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	26750
2	propose	ollama_remote	glm4:latest	0	0	13025
3	propose	ollama_remote	glm4:latest	0	0	12067
4	propose	ollama_remote	glm4:latest	0	0	16068
5	preregistration	ollama_remote	glm4:latest	0	0	9645
6	novelty	ollama_remote	glm4:latest	0	0	9789
7	novelty	ollama_remote	glm4:latest	0	0	8658

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18374
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18408
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12580
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12862
12	judge	ollama_remote	glm4:latest	0	0	11893

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 170118 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in [research/audit/4695f49f0b2b.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in [research/audit/4695f49f0b2b.jsonl](#) for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: [research/pvsnp_sandbox/test_*.py](#) (per-cycle)

Replay tarball: [research/replay/4695f49f0b2b.tar.gz](#) (if generated)